Supplementary Material: A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

This supplement collects the analyses behind the main paper. We give the full proofs of forward equivalence, the temperature-limit bound, and finite-parameter exactness; a numerical study of how α and T shape the gradient; measurements of forward bit-exactness under relaxation; throughput and active development time; and beam-timed comparison results. We first restate the execution modes and prove their properties.

1 Setup and Notation

The full VCS state is $x = (\text{CPU registers}, \text{RAM}, \text{RIOT}, \text{TIA}, \text{cartridge}, \text{frame buffer})$. The hard transition Φ_{H} executes one CPU instruction on the complete bit-exact machine. The full-system SOFT-STE transition Φ_{S} first executes this hard instruction, including all bus and device effects. It then replays the instruction through differentiable CPU/RAM handlers against the same hard machine. ROM bytes are read through the current bank/slice mapping of the hard cartridge. TIA, RIOT, and Superchip register reads are replayed as constants from the hard bus; writes outside RAM affect only the hard devices.

Every CPU register and RAM cell is returned as $\mathrm{STE}(\text{soft}, \text{hard})$. TIA, RIOT, cartridge, and frame-buffer states are carried by the hard machine and receive no gradient. The backward pass reaches CPU registers, RAM, and ROM bytes. We define five primitives for the differentiable handlers and relaxed execution.

The first primitive is the memory read. A read of a memory tensor $r \in \mathbb{R}^M$ of size M (the ROM or the RAM) at an integer address $a \in \{0, \dots, M-1\}$ is written as a dot product against the one-hot address indicator $\mathbf{1}_a$,

$$\mathrm{peek}(r, a) = \mathbf{1}_a^\top r = \textstyle\sum_i [\![i = a]\!]\, r_i = r_a, \tag{1}$$

so the forward value is the array element r_a, while the gradient with respect to r is the one-hot vector $\mathbf{1}_a$.

The second primitive is opcode dispatch. With a score vector (logits) $\ell \in \mathbb{R}^K$ over the $K = 256$ opcodes, a temperature $T > 0$, and the softmax weights $w = \mathrm{softmax}(\ell/T)$, the relaxed dispatch over the candidate handler-output rows V is the convex combination

$$\mathrm{select}(\ell, V; T) = w^\top V, \qquad w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}}. \tag{2}$$

This is a definition of a possible relaxation, not an executed dispatch mode. Every mode studied here uses the hard switch on the decoded opcode byte. No executed mode relaxes opcode dispatch, so it contributes no error term to the bounds below.

The third primitive is the conditional branch. For a relaxed status flag $f \in [0, 1]$ written as a logit $z = s(2f - 1)$, where the sign s selects branch-when-set or branch-when-clear, and a sharpness α, the gate and the relaxed program counter are

$$g = \sigma(\alpha z), \qquad \mathrm{pc}_{\mathrm{soft}} = (1-g)\,\mathrm{pc}_{\bar{b}} + g\,\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + g\,\delta, \tag{3}$$

where $\mathrm{pc}_{\bar{b}}$ is the fall-through (not-taken) counter and $\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + \delta$ is the taken counter for the signed branch offset δ.

The fourth primitive is the straight-through estimator. For any soft value that has a matching hard value it returns

$$\mathrm{STE}(\text{soft}, \text{hard}) = \text{soft} + \mathrm{sg}(\text{hard} - \text{soft}), \tag{4}$$

where $\mathrm{sg}(\cdot)$ is the stop-gradient operator, so the forward value equals *hard* while the backward pass uses the gradient of *soft*.

The fifth primitive is the *relaxed read*, the differentiable counterpart of the exact read (1). It blurs the integer address with a distance-softmax of width T, returning the temperature-weighted average of the neighbouring cells,

$$\mathrm{peek}_{\mathrm{R}}(r, a; T) = \textstyle\sum_k w_k\, r_{a+k}, \qquad w_k = \frac{e^{-|k|/T}}{\sum_j e^{-|j|/T}}, \tag{5}$$

so that $\mathrm{peek}_{\mathrm{R}} \to r_a$ as $T \to 0$ while the address carries a nonzero gradient for $T > 0$. The executed SOFT-STE path uses the exact one-hot read (1); FULL and LEAKY use (5) with their results rounded back to integers.

Table 1 summarises the six execution and gradient modes. FULL relaxes branches and reads only, with hard opcode dispatch. LEAKY is the same construction just outside the sufficient exactness region. NAIVE differentiates hard execution through discrete indices; SAMPLER adds a bilinear position kernel while retaining the hard forward.

Assumption 1 (Decision margins)**.** *Along the executed trajectory of length N, each relaxed conditional branch has strictly positive flag-logit margin $m_t = |z_t| > 0$. Let $m_{\min} = \min_t m_t > 0$.*

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Table 1: Execution and gradient modes. Only SOFT-STE replays the full-system instruction for CPU/RAM gradients.

Mode	Forward	Gradient	Used for
HARD	bit-exact	none	reference
SOFT-STE	= HARD	surrogate	CPU/RAM/ROM
FULL	exact in corner	relaxed	α/T study
LEAKY	mild divergence	relaxed	boundary test
NAIVE	bit-exact	zero on indices	baseline
SAMPLER	bit-exact	bilinear	position

For exact processor flags, $z_t = \pm 1$ and $m_{\min} = 1$. The relaxed read (5) needs no additional margin assumption: the addressed integer byte is separated from other addresses by distance at least one. Opcode dispatch is hard.

Numerical scope. IEEE-754 float32 represents every integer in $[-2^{24}, 2^{24}]$ exactly (IEEE 2019; Goldberg 1991). All VCS register values, data bytes, addresses, and instruction-local integer quantities lie within this range. The full-system SOFT-STE forward uses hard-machine values for every state component, so it does not accumulate soft arithmetic or rounding error. The exactness statement concerns these returned values; the soft computation supplies gradients.

2 Proofs

Theorem 1 (Exact forward equivalence)**.** *For every reachable full-machine state x and finite sharpness α and temperature $T > 0$, the full-system SOFT-STE step equals the hard step:*

$$\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x). \tag{6}$$

The complete trajectories therefore agree for every instruction under identical initial states and inputs. The modes differ only in their backward rules.

Proof. We first prove the one-step equality for an arbitrary reachable state x, then extend it by induction.

Hard instruction and soft replay. The full-system step first executes the instruction on the bit-exact hard machine. This updates the CPU registers, RAM, bus, TIA, RIOT, cartridge, and frame buffer exactly as Φ_{H} does. The selected opcode handler is determined by the hard decoded byte.

The soft CPU/RAM handlers then replay the instruction against that same hard machine. Opcode dispatch remains the hard switch. A ROM read uses the current hard-cartridge bank/slice mapping; a RAM read uses the soft tape. Reads of TIA, RIOT, and Superchip registers are constants from the hard bus, and writes outside RAM go to hard devices only. These rules ensure that the soft replay does not replace the hard device state with an approximation.

STE join. Let h_j be the hard result for any CPU register or RAM cell and s_j its soft replay result. The returned value is

$$\begin{aligned} \mathrm{STE}(s_j, h_j) &= s_j + \mathrm{sg}(h_j - s_j) \\ &= h_j. \end{aligned} \tag{7}$$

This equality holds for every value of s_j, without assuming that the soft handler numerically reproduces the hard one. Thus every returned CPU register and RAM cell is exactly hard. All other state components are already carried by the hard machine. Therefore every component of the complete state agrees, proving $\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x)$.

Floating-point equality. The hard values of VCS bytes, registers, addresses, and instruction-local counts lie within the exact integer range of float32. IEEE-754 single precision has a 24-bit significand and represents all integers in $[-2^{24}, 2^{24}]$ exactly (IEEE 2019; Goldberg 1991). For example, the one-hot read obeys

$$\mathrm{peek}_{\mathrm{S}}(r, a) = \mathbf{1}_a^{\top} r = r_a = \mathrm{peek}_{\mathrm{H}}(r, a). \tag{8}$$

The STE join likewise returns the hard integer value. Hence no representation error is introduced in the returned state. Equality is exact, not a tolerance comparison.

Induction on the trajectory. Let the two runs share x_0 and the same input sequence. For the base case, $x_0^{\mathrm{S}} = x_0^{\mathrm{H}}$. If $x_t^{\mathrm{S}} = x_t^{\mathrm{H}} =: x$, one-step equality gives

$$\begin{aligned} x_{t+1}^{\mathrm{S}} &= \Phi_{\mathrm{S}}(x) \\ &= \Phi_{\mathrm{H}}(x) = x_{t+1}^{\mathrm{H}}. \end{aligned} \tag{9}$$

By induction, the complete trajectories agree at every instruction.

Gradients. The stop-gradient changes the reverse-mode rule without changing the forward result. For example, the relaxed branch gate contributes $\alpha g(1-g)\delta$ to the derivative with respect to its logit, where the corresponding hard branch is discrete. The memory read (1) supplies the one-hot Jacobian $\mathbf{1}_a$ with respect to memory contents. The soft replay therefore exposes surrogate gradients through CPU registers, RAM, and ROM bytes. TIA, RIOT, cartridge, and frame-buffer states carry no gradient. These derivatives are not the derivatives of the hard discrete transition. □

Measured handler agreement. We tested the full-system soft step in both ports. In JUTARI, all 64 ROMs were checked starting 20 frames into play, over 3,000 instructions per game. SOFT-STE equalled HARD at every instruction (64/64), and the soft handlers before STE correction reproduced every hard CPU register and RAM cell, with zero disagreements.

In JAXTARI, nine ROMs cover every mapper family exercised by the conformance set: Pong and Breakout (2K), Space Invaders (4K), Ms. Pac-Man and Up'n Down (F8), Double Dunk (F6), Elevator Action (F8SC), Montezuma's Revenge (E0), and Robot Tank (FE). After standard boot and 20 frames of play, SOFT-STE equalled the hard console on all 1,500 instructions per ROM, with zero soft-handler disagreements. Furthermore, `jax.grad` through 100 full-system steps reaches ROM bytes.

The JUTARI check exposed two soft-handler defects, both fixed. The functional `BRK` handler did not push the required state and jump through the IRQ vector. Instruction fetches also always read ROM, incorrectly handling code executed from RAM, as in Up'n Down at $\mathrm{pc} = \$00\mathrm{EA}$. The tests therefore check the computation through which gradients pass, not only the STE result.

Theorem 2 (Temperature-limit bound)**.** *Consider FULL, which uses sigmoid branches* (3) *and distance-softmax*

reads (5) *without STE correction, while retaining hard opcode dispatch. Under Assumption 1, the deviations of the unrounded relaxed branch and read values from their hard counterparts satisfy*

$$\left\|\Phi_{\mathrm{S}}^{T,\alpha}(x) - \Phi_{\mathrm{H}}(x)\right\|_{\infty} \le C\left[e^{-\alpha m_{\min}} + \rho e^{-1/T}\right], \quad (10)$$

where the norm denotes the local deviations of these relaxed operations, C bounds their value ranges and fixed geometric factors, and ρ bounds the number of reads per step. The branch and read errors converge separately as $\alpha \to \infty$ and $T \to 0$:

$$\mathcal{O}(e^{-\alpha m_{\min}} + e^{-1/T}).$$

No opcode-dispatch term occurs.

Proof. We bound the two relaxed operations evaluated on the same hard input state. The bound concerns their unrounded values; rounding is addressed separately in Proposition 1.

Branch term. For a branch with margin $m = |z| \ge m_{\min}$, the difference between the sigmoid gate and the hard decision is

$$\begin{aligned}\left|\sigma(\alpha z) - [\![z > 0]\!]\right| &= \sigma(-\alpha m) \\ &\le e^{-\alpha m_{\min}}.\end{aligned} \quad (11)$$

Consequently, the unrounded program counter differs from the hard target by

$$\begin{aligned}\left|(g - [\![z > 0]\!])\delta\right| &\le |\delta| e^{-\alpha m_{\min}} \\ &\le C e^{-\alpha m_{\min}}.\end{aligned} \quad (12)$$

Read term. For an addressed byte r_a, the distance-softmax read gives

$$\mathrm{peek}_{\mathrm{R}}(r, a; T) - r_a = \sum_{k \ne 0} w_k (r_{a+k} - r_a). \quad (13)$$

Let w_0 denote the weight of the addressed byte. Since bytes lie in $[0, 255]$,

$$\left|\mathrm{peek}_{\mathrm{R}}(r, a; T) - r_a\right| \le 255(1 - w_0). \quad (14)$$

The denominator in (5) contains the addressed weight 1, so

$$\begin{aligned}1 - w_0 &\le 2 \sum_{d=1}^{\infty} e^{-d/T} \\ &= \frac{2e^{-1/T}}{1 - e^{-1/T}}.\end{aligned} \quad (15)$$

Thus

$$\left|\mathrm{peek}_{\mathrm{R}}(r, a; T) - r_a\right| \le \frac{510 e^{-1/T}}{1 - e^{-1/T}}. \quad (16)$$

For T bounded above by any fixed $T_0 > 0$, the denominator is bounded away from zero. Absorbing this fixed factor and the byte range into C gives the rate $Ce^{-1/T}$. The ρ reads contribute at most $\rho C e^{-1/T}$.

Combined bound. The executed opcode dispatch is a hard switch; its contribution is exactly zero. Adding the branch and read contributions yields

$$\delta_{\mathrm{step}} \le C\left[e^{-\alpha m_{\min}} + \rho e^{-1/T}\right], \quad (17)$$

which is (10). The two exponential rates remain separate; no relation between α and T is assumed. □

Proposition 1 (Finite-parameter exactness)**.** *FULL retains hard opcode dispatch and rounds each relaxed branch target and memory-read result to an integer. If every relaxed value differs from the corresponding hard integer by less than $1/2$, the rounded value equals the hard one. Hence the entire rounded trajectory is bit-exact.*

For a binary-flag branch with offset δ, the sufficient and, away from rounding ties, necessary condition is

$$|\delta|\sigma(-\alpha) < \frac{1}{2} \quad \Longleftrightarrow \quad |\delta| < \frac{1 + e^{\alpha}}{2}. \quad (18)$$

Since $|\delta| \le 128$ for a 6502 branch, $\alpha \ge 6$ suffices.

For an 8-bit distance-softmax read,

$$\begin{aligned}\left|\mathrm{peek}_{\mathrm{R}}(r, a; T) - r_a\right| &\le 255(1 - w_0) \\ &\le \frac{510 e^{-1/T}}{1 - e^{-1/T}} \\ &< \frac{1}{2} \qquad (T \le 0.14).\end{aligned} \quad (19)$$

Thus $\alpha \ge 6$, $T \le 0.14$ are program-independent sufficient conditions for the rounded FULL trajectory to equal HARD.

Proof. Branch casts. A hard branch with a binary flag chooses either $\mathrm{pc}_{\bar{b}}$ or $\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + \delta$. The sigmoid gate leaves weight $\sigma(-\alpha)$ on the unselected target. Therefore the absolute displacement of the relaxed result from the hard target is exactly

$$\epsilon_b = |\delta|\sigma(-\alpha). \quad (20)$$

Rounding to the nearest integer returns the hard target whenever $\epsilon_b < 1/2$. Using $\sigma(-\alpha) = 1/(1 + e^{\alpha})$,

$$\begin{aligned}\epsilon_b < \frac{1}{2} &\Longleftrightarrow \frac{|\delta|}{1 + e^{\alpha}} < \frac{1}{2} \\ &\Longleftrightarrow |\delta| < \frac{1 + e^{\alpha}}{2}.\end{aligned} \quad (21)$$

At $\alpha = 6$, the right-hand side is approximately 202.2, exceeding the maximum 6502 offset magnitude of 128. Hence all binary-flag branches round to their hard target.

Memory-read casts. For address a, let w_0 be the weight of the addressed byte. Each other byte differs from r_a by at most 255, so

$$\begin{aligned}\epsilon_r &= \left|\sum_{k \ne 0} w_k (r_{a+k} - r_a)\right| \\ &\le 255 \sum_{k \ne 0} w_k = 255(1 - w_0).\end{aligned} \quad (22)$$

For each distance $d \ge 1$, at most two addresses lie at that distance from a. Furthermore, the normalising denominator is at least one. Thus

$$\begin{aligned}1 - w_0 &\le 2 \sum_{d=1}^{\infty} e^{-d/T} \\ &= \frac{2e^{-1/T}}{1 - e^{-1/T}}.\end{aligned} \quad (23)$$

Reads near either end of the memory have fewer neighbours, which can only decrease this bound. It follows that

$$\epsilon_r \le \frac{510e^{-1/T}}{1-e^{-1/T}}. \tag{24}$$

At $T = 0.14$, the right-hand side is approximately $0.40 < 1/2$. The worst-case threshold is approximately $T = 0.1443$. Hence every relaxed byte read rounds exactly to its addressed hard value for $T \le 0.14$, regardless of the program or byte contents.

Induction over instructions. At the initial instruction the FULL and HARD states agree. Suppose they agree immediately before some instruction. Both decode the same opcode through the hard dispatch switch and execute the same non-relaxed operations. By the preceding bounds, every relaxed branch target and memory-read result rounds to its hard counterpart. Therefore the state after the instruction also agrees. Induction proves equality for the entire trajectory, without a Lipschitz or error-propagation assumption. □

The bounds are sufficient, not necessary: outside this corner, exactness depends on the branch offsets and byte patterns actually encountered. At $\alpha = 6$, $T = 0.14$, FULL was measured bit-exact for 6,000 instructions on Pong, Breakout, and Space Invaders. LEAKY ($\alpha = 5.5$, $T = 0.145$) lies just outside the sufficient region and exhibits mild forward divergence in the measured scene.

3 Effect of the Relaxation Parameters on the Gradient

Theorems 1 and 2 show that exact forward execution and useful gradients are distinct properties. We illustrate how α and T affect surrogate gradients in JUTARI, using a target invader sprite whose horizontal placement is controlled by the soft primitives.

For the sigmoid branch, we plot the screen-space directional derivative $\partial(\text{screen})/\partial(\text{control})$ while varying α (Figure 1). The colour encodes the sign of the derivative: red where a pixel brightens as the control increases, blue where it darkens. Intensity denotes magnitude. The colour bar gives absolute units; each panel also reports its peak relative to the row maximum.

For a position $\mathrm{pc} = (1-g)\mathrm{pc}_L + g\mathrm{pc}_R$, with $g = \sigma(\alpha z)$, the derivative is

$$\frac{\partial \mathrm{pc}}{\partial z} = \alpha g(1-g)(\mathrm{pc}_R - \mathrm{pc}_L). \tag{25}$$

This produces a dipole at the two sprite placements. Its magnitude is non-monotonic in α at fixed $z = 0.25$: small α gives a shallow gate, while large α saturates it. The measured relative peaks for $\alpha = 2, 6, 20$ are $0.53, 1.00, 0.15$, respectively. The intermediate $\alpha = 6$ gives the strongest gradient here.

The forward render in this STE gradient experiment remains the exact hard one (Theorem 1); only the surrogate derivative changes. The gradient vanishes as the relaxation approaches its hard limit. This is a property of the estimator, not evidence that its attributions are faithful.

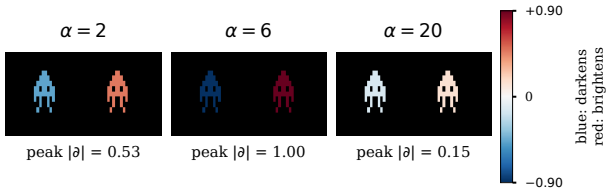


Figure 1: Soft branch: the screen-space gradient $\partial(\text{screen})/\partial z$ as sharpness α is swept in JUTARI. The gradient is a dipole: blue where a sprite leaves, red where it arrives. The colour bar is in absolute units (here ± 0.90); panel values are relative peaks. The gradient is strongest at $\alpha = 6$ and nearly vanishes at $\alpha = 20$ as the gate saturates. The STE forward remains bit-exact.

4 Forward Bit-Exactness Under Full Relaxation

SOFT-STE is forward-exact independently of α and T (Theorem 1). FULL instead relaxes conditional branches and memory reads without STE correction, retaining hard opcode dispatch. Every relaxed branch target and byte read is rounded back to an integer. Proposition 1 proves that $\alpha \ge 6$ and $T \le 0.14$ suffice for exactness independently of the executed program.

We test the rounding boundaries on the full JUTARI simulator executing Space Invaders. The hard, cycle-timed TIA and other devices remain unchanged; only the CPU's branch and read operations pass through the relaxation hook.

A crisp-flag branch with signed offset δ rounds to the hard target when $|\delta| < \tau_b(\alpha)$, where $\tau_b(\alpha) = \frac{1}{2}(1 + e^{\alpha})$. The empirical per-branch exactness $p_{\mathrm{branch}}(\alpha)$ is the fraction of executed offsets satisfying this condition.

A relaxed read is exact when its neighbour pull

$$\left| \sum_{k \ne 0} w_k \big(\mathrm{mem}[a+k] - \mathrm{mem}[a]\big) \right| < \frac{1}{2}. \tag{26}$$

The corresponding empirical fraction is $p_{\mathrm{read}}(T)$. Treating individual casts as independent gives the heuristic per-step likelihood

$$P_{\mathrm{step}}(\alpha, T) = p_{\mathrm{read}}(T)^{\rho} p_{\mathrm{branch}}(\alpha)^{f_b}. \tag{27}$$

For Space Invaders, the executed trace has $\rho = 2.16$ reads per instruction and branch fraction $f_b = 0.374$. Independence is an approximation, so P_{step} is not a probabilistic guarantee. Exactness of a particular trajectory requires every cast to round correctly; Proposition 1 supplies sufficient conditions for this without the likelihood model.

Figure 2 shows a bit-exact corner at large α and small T, with a gradual branch boundary and a sharp read boundary. Table 2 measures the first divergence rather than the final state, since game-state reinitialisation can partially resynchronise divergent trajectories.

At $T = 0.14$, $\alpha = 3, 4, 5$ first diverge at instructions $879, 1121, 1216$, respectively; $\alpha = 6$ remains exact. At $\alpha = 6$, the read-exact fraction falls from 1 at $T \le 0.14$

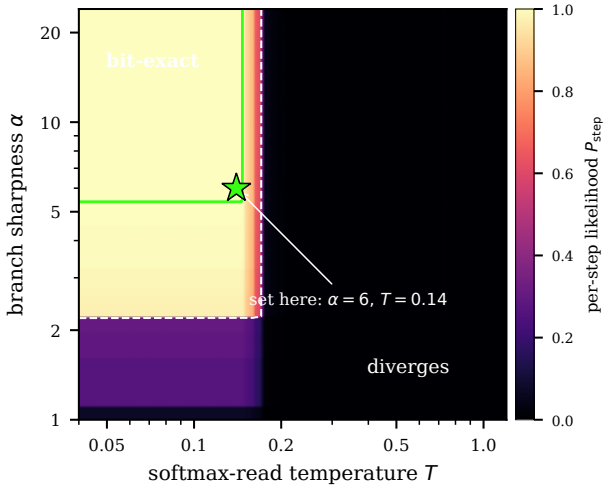


Figure 2: Heuristic per-step exactness likelihood $P_{\text{step}} = p_{\text{read}}(T)^{\rho} p_{\text{branch}}(\alpha)^{f_b}$ for Space Invaders. The green contour marks $P_{\text{step}} = 1$; the dashed white contour marks 0.5. The star marks the sufficient program-independent setting $\alpha = 6$, $T = 0.14$ (Proposition 1). The likelihood model describes the measured boundary, not the source of the exactness guarantee.

to 0.981 at $T = 0.15$ and 0.144 at $T = 0.20$. The measured boundary agrees with the cast-margin analysis. The program-independent guarantee, however, comes from Proposition 1, not the observed P_{step} values. The same cast-to-integer logic is used in JAXTARI.

Choosing α and T. Proposition 1 gives program-independent sufficient conditions. For 6502 branch offsets $|\delta| \le 128$, $\alpha = 6$ gives $\frac{1}{2}(1 + e^{\alpha}) \approx 202.2 > 128$. For byte reads, the worst-case pull $510e^{-1/T}/(1 - e^{-1/T})$ is approximately 0.40 at $T = 0.14$, below the rounding threshold $1/2$. The exact sufficient temperature threshold is approximately 0.1443. Thus $\alpha = 6, T = 0.14$ guarantees bit-exact rounded branch and read operations under the stated model.

Measurements confirm bit-exact execution over 6,000 instructions on Pong, Breakout, and Space Invaders at this setting. Outside the sufficient corner, failure depends on the executed program: $\alpha = 5$ diverges on Space Invaders but not Pong or Breakout; $T = 0.15$ diverges on Pong and Space Invaders but not Breakout. The selected setting lies near the finite-parameter boundary, retaining a useful relaxation gradient while preserving the exact forward. More extreme parameters saturate the branch and read gradients.

Full-simulator demonstration. Figure 3 compares four beam-timed Space Invaders rollouts over 60 seconds. HARD, SOFT-STE, and FULL at $\alpha = 6, T = 0.14$ render identical frames. The relaxed executions use the same cycle-accurate hard renderer with a default-off relaxation hook for CPU branches and reads. LEAKY at $\alpha = 5.5, T = 0.145$ diverges mildly but remains recognisable, with a persistent missing invader column caused by a corrupted sprite-position read.

Table 2: Forward exactness of FULL on Space Invaders, over 3,000 instructions from reset. Measured N_{div} is the first divergent instruction; "exact" means none. Predicted columns are cast fractions and heuristic per-step likelihood, with $\rho = 2.16$, $f_b = 0.374$.

		Meas.	Predicted		
α	T	N_{div}	p_{branch}	p_{read}	P_{step}
Branch boundary ($T = 0.14$, read-exact)					
2	0.14	7	0.039	1.000	0.298
3	0.14	879	0.953	1.000	0.982
4	0.14	1121	0.988	1.000	0.996
5	0.14	1216	0.996	1.000	0.999
6	0.14	*exact*	1.000	1.000	1.000
Temperature boundary ($\alpha = 6$, branch-exact)					
6	0.08	*exact*	1.000	1.000	1.000
6	0.10	*exact*	1.000	1.000	1.000
6	0.12	*exact*	1.000	1.000	1.000
6	0.14	*exact*	1.000	1.000	1.000
6	0.15	803	1.000	0.981	0.959
6	0.18	2	1.000	0.223	0.039
6	0.20	2	1.000	0.144	0.015

Delayed divergence. A relaxed run can boot exactly and diverge only after the game state changes. RAM neighbour values evolve, and a read may cross its rounding threshold only after many frames. For a clean-boot rollout with relaxation enabled afterward and $\alpha = 20$, the first divergent frame is 1 at $T = 0.1448$, 194 at $T = 0.1446$, and 2365 (approximately 40 seconds) at $T \le 0.1444$. Below $T = 0.1465$, the lowest fixed ROM-read threshold, opcode and operand fetches remain exact; these late failures come from evolving RAM reads. The boundary is therefore a spectrum of fixed ROM and state-dependent RAM thresholds.

5 Per-Game Conformance

Table 3 lists every game in the conformance set with its auto-detected mapper, a short ROM hash, and its per-frame RAM and pixel exactness, so the headline 64/64 claim can be audited game by game. Both ports pass every game; JAXTARI and JUTARI agree game-for-game, and the cross-check between them is part of the test suite.

The conformance protocol uses the standard ALE boot (60 NOOP frames and four resets), followed by a fixed action stream. RAM is checked on every one of 30 frames and the complete 210×160 frame buffer on every one of 60 frames. The results establish exactness on these tested trajectories, not on all possible executions. Ten cartridge mappers are implemented, seven exercised by the 64 games.

Opcode coverage. jutari implements 189 opcode bytes: 151 documented, the undocumented USBC alias (`$EB`), and 37 illegal (27 multi-byte NOP variants, 6 LAX, 4 SAX); 67 bytes lack handlers. Counting every executed instruction across 64 conformance ROMs, boot plus the 60-frame window covers 134/151 documented opcodes in 78.8 million instructions; boot plus 3,600 steps covers 135/151 in 1.21 billion. Neither run executes an illegal opcode, USBC, or an

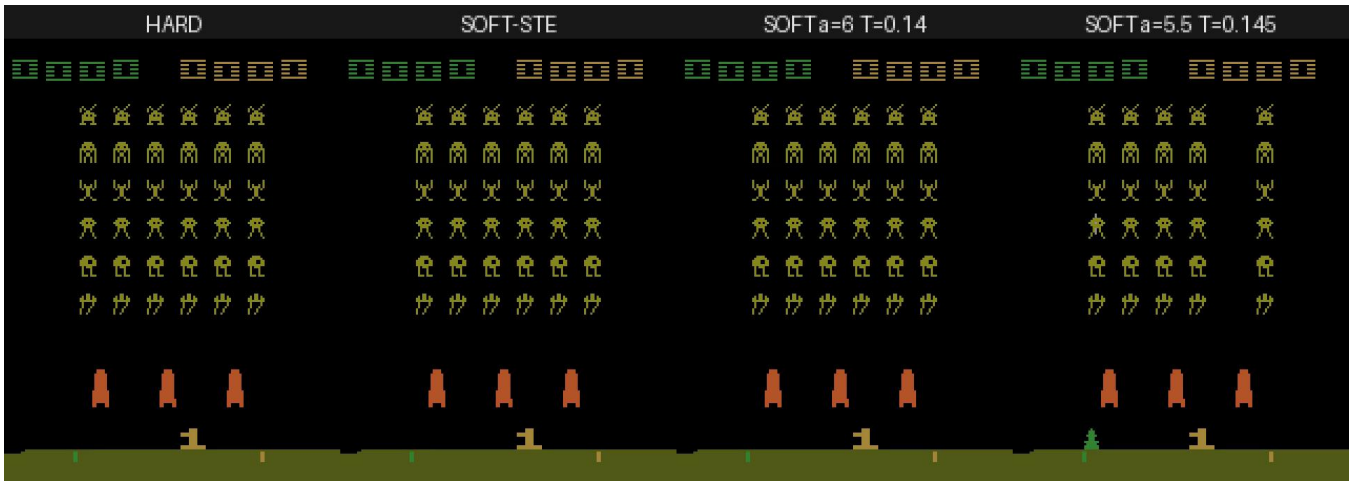


Figure 3: One frame of the 60-second beam-timed Space Invaders comparison. HARD and SOFT-STE are pixel-identical. FULL ($\alpha = 6, T = 0.14$) remains identical throughout the rollout. LEAKY ($\alpha = 5.5, T = 0.145$) diverges mildly: a column of invaders is missing after a sprite-position read rounds incorrectly.

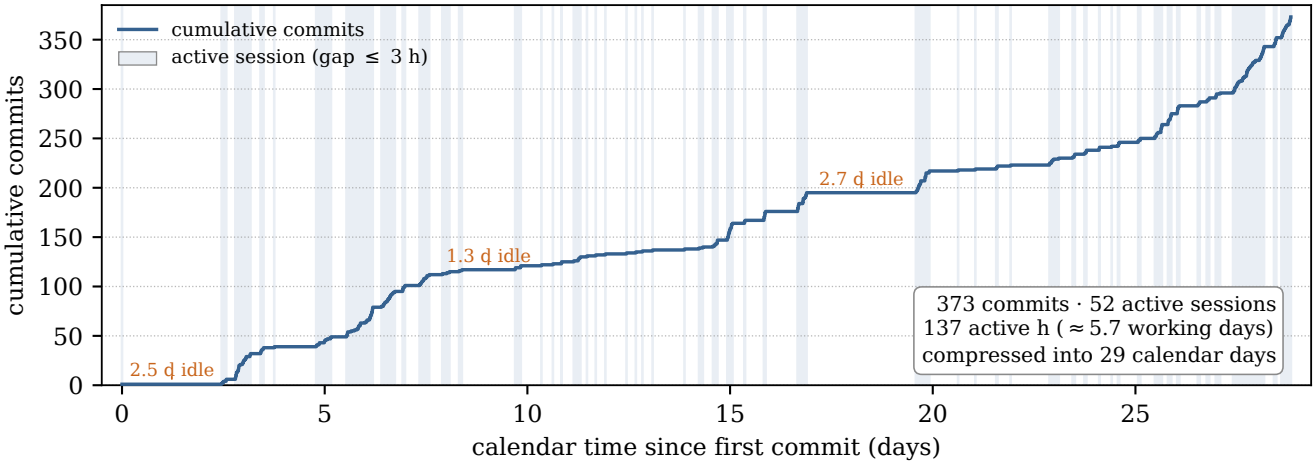


Figure 4: Active development time reconstructed from the version-control log. The curve shows cumulative commits; shaded bands denote sessions separated by at most three hours between commits. The 373 commits form 52 sessions totalling 136.8 hours over a 29-day calendar window. This is estimated development wall-time, not person-effort or compute.

Table 3: Per-game conformance over the 64 ALE games supported by our XITARI configuration. **Map**: mapper auto-detected by the ports (the set exercises seven of the ten implemented formats: 2K, 4K, F8, F6, E0, FE, F8SC; F4, F6SC and F4SC are implemented but unused here). **SHA-256**: first 12 hex digits of the ROM image (full hashes ship in the repository manifest; ROMs are not redistributed). **R/P**: RAM- and pixel-exact against XITARI on every evaluated frame (30 NOOP frames for RAM, 60 frames of the fixed action stream for pixels, after the standard 60-NOOP+4-reset boot). All 64 games are exact in both (✓); JAXTARI matches JUTARI game-for-game.

Game	Map	SHA-256	R	P
air_raid	4K	6b5a22ec074a	✓	✓
alien	4K	9cd556c7d7f5	✓	✓
amidar	4K	da5b760d3ada	✓	✓
assault	4K	d32ca4dd9e84	✓	✓
asterix	F8	b6161db0530d	✓	✓
asteroids	F8	d1a6e808412b	✓	✓
atlantis	4K	cef4b7b91ea2	✓	✓
bank_heist	4K	c32daffdb926	✓	✓
battle_zone	F8	efd6f639e724	✓	✓
beam_rider	F8	83b30cb52649	✓	✓
berzerk	4K	cdff0422329d	✓	✓
bowling	2K	dff44a85289f	✓	✓
boxing	2K	54760d7e0710	✓	✓
breakout	2K	376323f051c3	✓	✓
carnival	4K	80eab759b6bd	✓	✓
centipede	F8	6def669f54bd	✓	✓
chopper_command	4K	055637282252	✓	✓
crazy_climber	F8	486ad4cc8956	✓	✓
defender	4K	e3c7ed7a073f	✓	✓
demon_attack	4K	8d72feeb267d	✓	✓
double_dunk	F6	c0ea53a91a39	✓	✓
elevator_action	F8SC	f3b3ad03fd8c	✓	✓
enduro	4K	ca9bb89755a6	✓	✓
fishing_derby	2K	84753bc6ecb4	✓	✓
freeway	2K	3ef620234b98	✓	✓
frostbite	4K	cbfdad89480d	✓	✓
gopher	4K	b4aff03aeb0f	✓	✓
gravitar	F8	17a19cd7b5ca	✓	✓
hero	F8	75bd02ec7545	✓	✓
ice_hockey	4K	06df1fc568d9	✓	✓
jamesbond	E0	995253f0d3a2	✓	✓
journey_escape	4K	963e2d3da52b	✓	✓
kangaroo	F8	a2826eaea3a8	✓	✓
krull	F8	b40f10d14217	✓	✓
kung_fu_master	F8	3f6501a649ad	✓	✓
montezuma_revenge	E0	69d363a74754	✓	✓
ms_pacman	F8	dde0b43c5dee	✓	✓
name_this_game	4K	cc219a7246e8	✓	✓
pacman	4K	58e781b472e0	✓	✓
phoenix	F8	b01085bc5227	✓	✓
pitfall	4K	c719b47714d8	✓	✓
pong	2K	41623e3c2614	✓	✓
pooyan	4K	ac9abbfc272d	✓	✓
private_eye	F8	c8f9ce1b2e80	✓	✓
qbert	4K	3257221832a7	✓	✓
riverraid	4K	4c6842b8af64	✓	✓
road_runner	F6	4b6c2e68d693	✓	✓
robotank	FE	c186409481e6	✓	✓
seaquest	4K	fbc29f4678f6	✓	✓
skiing	2K	804eff5f0196	✓	✓
solaris	F6	0afa36e5f4d8	✓	✓
space_invaders	4K	7224b17462b9	✓	✓
star_gunner	4K	3ad501a39280	✓	✓
surround	2K	c6864f5c9f43	✓	✓
tennis	2K	5a2052f020bf	✓	✓
time_pilot	F8	8ea97e335a2f	✓	✓
tutankham	E0	403397fe8583	✓	✓
up_n_down	F8	74890a43e591	✓	✓
venture	4K	1870b0dd6fb1	✓	✓
video_pinball	4K	cef82c39bbb0	✓	✓
videochess	4K	d44f2a115393	✓	✓
wizard_of_wor	4K	d69ee89e65c3	✓	✓
yars_revenge	4K	ff777c8d4ea0	✓	✓
zaxxon	F8	e1c323ce00cc	✓	✓

Table 4: Soft CPU core throughput (Pong, 3,000 instructions). The simplified bus has no TIA, one 4 KB ROM window, and only the RIOT timer. Rates are aggregate instructions/s.

Engine and backend	Mode	Instructions/s
JUTARI, CPU	single env	370,100
JAXTARI, CPU	single env	1,178
JAXTARI, CPU	batched (`vmap`)	∼60,000
JAXTARI, GPU	batched (`vmap`)	**3,119,115**

unhandled byte. Thus, 64/64 conformance does not test illegal opcodes, which have unit-test coverage only. The 16 unexecuted documented opcodes are indexed-indirect, indirect-indexed, and absolute read-modify-write forms. The script and per-game table ship with the code.

Long horizon. We compared jutari against seed-patched XITARI over all 3,600 frames (60 times the conformance window), using the same harness, boot, settings, and action stream, without game-over auto-reset. RAM is bit-exact on every frame for 63/64 games; screens are pixel-exact for 62/64. All three exceptions are transient and re-converge: `amidar` differs in one RAM byte (stacked processor status, I flag) at frame 392, with an exact screen; `alien` has two whole-frame screen differences (first at 526), with RAM exact throughout; `carnival` differs by one pixel on two frames (first at 767). Earlier divergences in 13 games arose from the comparison-video pipeline's game-over/auto-reset wrapper, not the emulator; those games are exact here. The sweep script and table ship with the code.

Reference determinism. XITARI fills its 128-byte Superchip RAM at power-on using a pseudo-random generator seeded with `time(NULL)`, independently of the emulator seed. This makes games reading RAM before writing it (e.g., `elevator_action`) non-reproducible. Our patch seeds the generator with zero; both ports reproduce the resulting byte sequence. A second patch adds a bus-trace hook for debugging. Both ship with the code.

6 Throughput

One step is one CPU instruction, not an ALE frame. We distinguish the soft CPU core, used for the batched JAXTARI measurements, from the full-system SOFT-STE step.

The soft CPU core has a simplified bus: no TIA, a single 4 KB ROM window, and only the RIOT timer. It excludes the complete TIA, RIOT, and cartridge execution of the full-system step. The JAXTARI hard interpreter remains eager and sequential. Record-and-replay enables compiled, GPU-batched full-system SOFT-STE differentiation, but GPU throughput excludes the one-time hard recording cost per trajectory.

CPU measurements use an Apple M1 Max core (Julia 1.12, JAX CPU backend). GPU measurements use a GTX 1080 Ti and a Quadro RTX 5000. JIT warm-up is excluded. Table 4 summarises soft-core rates; Tables 5 and 6 give batch scaling.

Table 5: Batched JAXTARI soft CPU core throughput on Apple M1 Max (Pong, compiled `lax.scan` with `vmap`). Aggregate rates count CPU instructions across environments; per-env is aggregate$/N$.

batch N	aggregate (instr./s)	per-env (instr./s)	vs. $N = 1$
1	30,200	30,200	1.0×
16	35,200	2,200	1.2×
64	48,300	750	1.6×
128	55,900	440	1.9×

Table 6: Batched JAXTARI soft CPU core on one GTX 1080 Ti (Pong, 3,000 instructions, `lax.scan` and `vmap`; mean ± standard deviation over 10 timed runs). Forward+gradient is approximately 5% slower than forward at the peak near $N = 4096$. Neither includes full-system TIA/RIOT execution.

batch N	forward (instr./s)	forward+grad (instr./s)
1	$2{,}458 \pm 1$	$1{,}265 \pm 4$
64	$258{,}747 \pm 17$	$224{,}194 \pm 48$
256	$677{,}299 \pm 345$	$595{,}776 \pm 177$
1024	$2{,}024{,}825 \pm 216$	$1{,}819{,}602 \pm 365$
4096	$\mathbf{2{,}949{,}204 \pm 2{,}562}$	$\mathbf{2{,}799{,}834 \pm 162}$
16384	$2{,}848{,}876 \pm 490$	$2{,}575{,}793 \pm 444$
65536	$2{,}599{,}829 \pm 100$	$2{,}534{,}157 \pm 43$

Soft CPU core. At single-instruction granularity, JUTARI reaches approximately 370,100 instructions/s and JAXTARI 1,178. JAX incurs substantial per-operation overhead at this granularity. Compiling a 3,000-instruction Pong rollout into `lax.scan` increases single-environment JAXTARI throughput to approximately 30,000 instructions/s. A 50,000-instruction trace takes approximately 42 seconds for one forward-plus-backward pass (gradient) through the JAXTARI soft CPU core with its simplified bus, as used for one attribution. This is not the full-system step.

Batching with `vmap` increases aggregate CPU throughput toward 60,000 instructions/s. The 256-way `lax.switch` evaluates candidate branches under batching, limiting CPU scaling but allowing effective parallel evaluation on GPU.

GPU. On the GTX 1080 Ti, forward throughput rises from approximately 2,500 instructions/s at $N = 1$ to 2.95 million at $N = 4096$, approximately 50 times the CPU batching limit, then declines slightly at larger batches. Forward+gradient reaches approximately 2.80 million instructions/s at the peak. The workload uses approximately 0.1 GB there, so available parallelism rather than memory capacity limits performance.

The Quadro RTX 5000 shows the same scaling pattern, peaking at $3{,}117{,}908 \pm 2{,}871$ instructions/s (mean and standard deviation over 10 runs) near $N = 4096$. The GPU results establish efficient batched soft CPU core execution, not differentiable full-machine rendering.

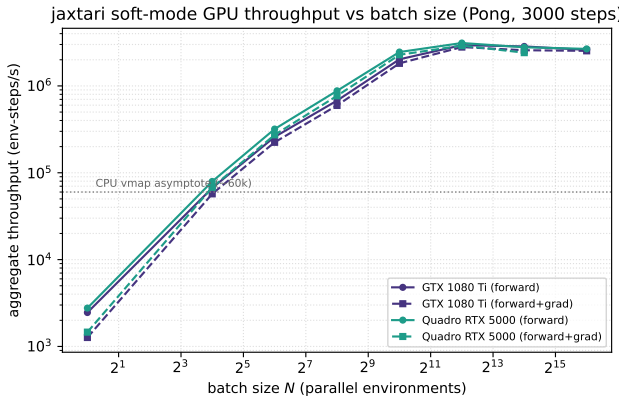


Figure 5: JAXTARI soft CPU core throughput versus batch size on GTX 1080 Ti and Quadro RTX 5000 (Pong, 3,000 instructions). Forward and forward+gradient peak near $N = 4096$ at approximately 3 million aggregate instructions/s. The dotted line indicates the CPU batching limit. These are simplified-bus results, not full-system differentiable rollouts.

Full-system soft step. Both ports implement the full-system SOFT-STE construction: the hard machine executes CPU, bus, TIA, RIOT, and cartridge effects; soft handlers replay the instruction; CPU registers and RAM are joined with STE. In JUTARI, the step reaches approximately 146,000 instructions/s on Pong using one Apple M1 Max core (median of three runs). In JAXTARI, eager execution reaches 943 forward and 0.13 gradient instructions/s; record-and-replay (below, Table 7) avoids this cost.

Zygote reverse-mode gradients through this full-system step depend strongly on window length. Measured throughput is approximately 400–450 instructions/s over 50–100-instruction windows, 250 instructions/s at 200 instructions, and 120 instructions/s at 400 instructions. It falls further for longer windows. Both the full-system and older soft-core reverse passes slow down superlinearly with window length under Zygote. The soft core is approximately 1.5 times faster at 50 instructions, 3.7 times at 100, 4.8 times at 200, and 7 times at 400 instructions.

In JUTARI, Zygote gradients remain practical only for short windows; JAXTARI replay has flat gradient cost across window lengths. An efficient JUTARI reverse pass remains future work.

Full-system replay. The eager hard console records bank/slice mappings, TIA/RIOT/Superchip reads, and hard register/RAM values for each instruction. A compiled `lax.scan` replays SOFT-STE with `jit`, `vmap`, and `grad`. Replay forward matches eager SOFT-STE and hard execution; ROM gradients are bitwise identical to eager gradients. Recording runs at 2,306 instructions/s on an M1 Max core and is slower on cluster CPUs. Single-trajectory CPU replay reaches 133k–179k forward and 41k–75k gradient instructions/s; gradient cost is flat across 100, 400, and 3,000 instructions. Single-trajectory GPU replay is slower (9k–16k forward, 3.6k–5.7k gradient), owing to launch latency. Batched throughput uses a 400-instruction trace; the 256-way opcode switch evaluates all branches. At $B = 1024$, 16 GB GPUs require 20-instruction checkpoints instead of 100; $B = 4096$ gradients exceed 48 GB (Table 7).

Table 7: Full-system SOFT-STE replay throughput on a recorded 400-instruction Pong trace, batched over B copies. Forward and gradient rates are aggregate environment-instructions/s, excluding sequential hard recording. Gradient checkpoint intervals are 100 instructions, except 20 at $B = 1024$ on 16 GB GPUs.

GPU	memory	B	forward (instr./s)	gradient (instr./s)
V100	16 GB	256	507,000	69,000
RTX 5000	16 GB	256	828,000	81,000
A6000	48 GB	256	954,000	83,000
V100	16 GB	1024	1,630,000	177,000
RTX 5000	16 GB	1024	2,010,000	179,000
A6000	48 GB	1024	**2,620,000**	**254,000**

7 Active Development Time

Figure 4 gives the version-control evidence for the active development time estimate. All figures derive from the project's `git` history, which will be released with the code. Commit timestamps provide a wall-time proxy, not a direct measurement of person-effort, coding-agent compute, or total labour.

We group commits into sessions separated by gaps greater than three hours. This exceeds the median in-session gap of approximately 13 minutes. A two-hour cutoff gives 106 hours; a four-hour cutoff gives 162 hours, compared with 136.8 hours at three hours.

The 373 commits form 52 sessions across 29 calendar days, totalling approximately 5.7 round-the-clock days of active development wall-time. Around 80% of the calendar span is excluded as idle, including travel without reliable internet access. The figure is an order-of-magnitude estimate, not a controlled comparison with conventional development.

8 Fixed Settings and Parameters

Table 8 consolidates the fixed settings and relaxation parameters used across the paper's experiments. The executed gradient mode is SOFT-STE, whose forward pass is bit-identical to HARD and is therefore independent of the temperature T; the sharpness α and T enter only the relaxation analysis and the surrogate gradient.

Table 8: Fixed settings and (hyper-)parameters used in the paper's experiments.

Setting	Value
Soft sharpness α	2, 6, 20 (gradient-shape study)
Soft temperature T	swept; SOFT-STE forward independent of T
Numeric representation	float32, integer-valued in $[-2^{24}, 2^{24}]$
Boot sequence	60 NOOP + 4 RESET frames
Episode-start randomization	0–30 random NOOPs (Mnih-style)
Conformance window	30 frames (RAM), 60 frames (screen)
Throughput rollout length	3,000 CPU instructions
Throughput batch sizes N	1, 16, 64, 256, 1024, 4096, 16384, 65536
Throughput repeats	10 (GPU), 3 (CPU baseline); JIT warm-up excluded

References

Goldberg, D. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, 23(1): 5–48.

IEEE. 2019. IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019). IEEE Std 754-2019.